Aug 21

Quick recap

AlgoCamp conducted a detailed session on the REACT (Reasoning and Acting) pattern for building AI agents, explaining how this pattern addresses fundamental limitations of LLMs by combining reasoning capabilities with action-taking abilities. The discussion covered the core components of the REACT loop, including how agents interact with tools to perform file operations, terminal commands, and other system interactions, while incorporating human-in-the-loop safety measures to prevent harmful actions. AlgoCamp introduced several open-source coding agents like Klein and Pi as reference implementations, and outlined plans to build a Claude Code-like agent using LangChain framework in upcoming sessions. The session concluded with an overview of Hugging Face as an alternative model platform and a homework assignment for students to build a simple coding agent using plain Python before moving to framework-based implementations.

Next steps

AlgoCamp

- Add a formal homework assignment for attendees to build a simple Claude Code-like coding agent using a plain Python agent loop, with CRUD on the file system and the ability to run terminal commands.
- Share the GitHub repository containing the code discussed in the session.
- Prepare and share an introductory discussion on open-source coding agents (e.g., Klein, Pi) and tools like DeepWiki for understanding their architectures.
- Start the next session with an introduction to message queues, basics of Temporal, and begin building a coding agent using LangChain.

Collaboration

- Attendees: Attempt to build a simple coding agent using the plain Python agent loop, incorporating file system CRUD operations and terminal command execution, considering security aspects and potential for human-in-the-loop approvals.

Summary

LLM Agents and REACT Framework

AlgoCamp led a session on the basics of agents, reviewing previous work on simple agents and introducing the concept of a harness around LLMs to enable more complex tasks. The discussion focused on the REACT paper, which explores using LLMs for both reasoning and acting in an interleaved manner to improve context and enable actions like code generation and tool interaction. AlgoCamp explained how LLMs can signal which tools to use and how to interface with external sources, emphasizing the synergy between reasoning and acting in agent development.

REACT Pattern for LLM Interactions

AlgoCamp explained the REACT pattern, which stands for Reasoning plus Acting, as a way for language models to interface with the environment through tool calls rather than direct control. The discussion highlighted how REACT improves performance on tasks like question answering and decision-making by allowing LLMs to interact with APIs like Wikipedia, reducing hallucinations and error propagation. AlgoCamp emphasized that while LLMs can generate token sequences, they cannot directly access external systems like code repositories or databases, making tool-based interactions essential for providing additional context to the models.

Cloud Coding Agent Development Discussion

AlgoCamp explained the concept of agent patterns and discussed tools needed for creating a cloud coding agent like Claude Code. The proposed tools include file system operations (create, edit, delete, list files), command execution capabilities, and additional features like web search and bug detection across code repositories. AlgoCamp also explained the difference between traditional LLMs and reasoning-based models, noting that reasoning-based models like O1 and R1 generate intermediate tokens to break down complex problems into smaller subproblems before producing final outputs.

Reasoning-Based LLMs Demonstration

AlgoCamp explained reasoning-based LLMs, which are AI models designed to spend extra time thinking through problems before providing answers, unlike traditional models that rely solely on next token prediction. They demonstrated how these models break down complex queries into intermediate logical steps and evaluate multiple paths before giving a final answer. AlgoCamp also described the React loop pattern, explaining how reasoning-based models interact with their environment through observation and action, allowing them to gather additional context and make informed decisions before generating final outputs.

React Pattern for AI Agents

AlgoCamp discussed the React pattern for AI agents, explaining how it combines reasoning and acting to improve task performance. The pattern was demonstrated through examples of searching for information about Apple remote and Front Row software, showing how an agent can use a combination of search tools and logical reasoning to find answers. AlgoCamp explained that React is superior to simple acting or chain-of-thought approaches, as evidenced by performance observations showing React outperforms Act consistently and performs well across different domains including question answering and fact verification.

LLM Prompting Strategies Discussion

AlgoCamp discussed the performance of different LLM prompting strategies, explaining that React performs best for fine-tuning while Chain of Thought performs best for prompting LLMs. The discussion covered the importance of running code in isolation, particularly for testing purposes, and introduced the concept of Human-in-the-Loop (HITL) as a safety mechanism for critical decisions. AlgoCamp emphasized that modern coding agents should include verification steps where humans can approve actions before execution, rather than allowing autonomous execution of potentially harmful commands.

Human-in-the-Loop Agentic Infrastructure

AlgoCamp discussed the concept of human-in-the-loop (HITL) interactions in agentic infrastructure, using examples like password-protected files requiring human intervention. The discussion explored challenges with long-running tasks in coding agents, including context management and handling failures or timeouts. Participants considered solutions like sharing session history between chats and using external tools like .md files to track progress. The conversation ended with plans for a break before continuing the discussion on related topics.

LangChain Learning Plans Discussion

AlgoCamp discussed plans to start learning about LangChain next week by building a coding agent as their first project. The team will begin with an MVP and gradually add features like long running tasks and sandboxed environments. AlgoCamp clarified that their current work focuses on the application and inference layers, not the LLM layer, explaining that inference involves using fixed model weights without making changes to the model itself.

Coding Agents Development Discussion

AlgoCamp discussed the development of coding agents, focusing on existing open-source tools like Klein and Pi coding agents, and introduced DeepWiki as a resource for exploring these repositories. He explained the importance of human-in-the-loop safety mechanisms and tool capabilities in coding agents, emphasizing the need for proper approval flows for side-effecting actions. The discussion covered the use of frameworks like LangChain for building agents, alternatives such as CrewAI and AgentX, and the role of message queues like Temporal for handling long-running tasks. AlgoCamp assigned a homework task to create a simple coding agent using a plain Python agent loop with CRUD file system operations and terminal command execution, while emphasizing the importance of security measures to prevent access to sensitive data.